

[5]

Size ≥ 0 — 4

Array basic

int [] a = {10, 20, 30, 40, 0, 0, 0, 0};
 int size = 4

① insert in the middle $\Rightarrow$ shifting $O(n)$
 insert 15 on 1st index.

size = 4 index = 1

~~is size~~ void insertAt (, , ,) {
 for (int i = size; i > index; i--) {

 a[i] = a[i-1]

 shifts++;

}

 a[index] = value

}

i = 4 index = 1 $\Rightarrow$ 4 > 1 ✓ shift 1

i = 3 index = 1 $\Rightarrow$ 3 > 1 ✓ shift 2

i = 2 index = 1 $\Rightarrow$ 2 > 1 ✓ shift 3

i = 1 index = 1 $\Rightarrow$ 1 > 1 ✗

then it a[index] = value

a[1] = 15;

② delete element at index $O(n)$

delete 20

void Delete (, , ,) {

 for (int i = index; i < size - 1; i++) {

 a[i] = a[i+1]

 shifts++;

 a[size-1] = 0;

 i = index

i = 0 size = 4 $\Rightarrow$ 0 < 4-1 ✓ shift 1

i = 1 size = 4 $\Rightarrow$ 1 < 4-1 ✓ shift 2

i = 2 size = 4 $\Rightarrow$ 2 < 4-1 ✓ shift 3

i = 3 size = 4 $\Rightarrow$ 3 < 4-1 ✗

a[size-1] = 0;

Stack* Insertion in stack (Push)

```

void push (int value) {
    if (top == stack.length - 1) {
        S.O.P ("stack overflow");
        return;
    }
    stack[++top] = value;
    S.O.P (value + " pushed into stack");
}

```

top = -1, capacity = 4

-1 = 4-1 X

stack[0] = value (10) Insert 10

0 = 4-1 X

stack[1] = value (20) 1 20

1 = 4-1 X

stack[2] = value (30) 2 30

2 = 4-1 X

stack[3] = value (40) 3 40

3 = 4-1 X ✓

~~stack[4] = value (50)~~

or P = stack overflow.

* Deletion from stack (Pop)

```

if (isEmpty()) {

```

```

    S.O.P ("stack underflow");

```

```

    return -1; }

```

```

    return stack[top--];

```

top = 3

2

1

0

-1

top--

3

2

1

0

* Display stack elements

```
for (int i = top; i >= 0; i--) {
    s.o.p(stack[i]);
}
```

$i = 3 \Rightarrow 40$ $i >= 0 \checkmark \Rightarrow 40$
 $i = 2 \Rightarrow 30$ $i >= 0 \checkmark \Rightarrow 30$
 $i = 1 \Rightarrow 20$ $i >= 0 \checkmark \Rightarrow 20$
 $i = 0 \Rightarrow 10$ $i >= 0 \checkmark \Rightarrow 10$

* Queue

* Enqueue in Queue

```
if (rear == Max - 1) {
    s.o.p("overflow");
}
```

```
if (front == -1) {
    front = 0;
}
```

```
Queue[rear] = value
```

rear = -1, front = -1, Max = 4

-1 == 3 X

Queue[0] = 10 ——— insert 10

0 == 3 X

Queue[1] = 20 ——— insert 20

1 == 3 X

Queue[2] = 30 ——— insert 30

2 == 3 X

Queue[3] = 40 ——— insert 40

3 == 3

✓

stop here because
queue is overflow.

10	20	30	40
----	----	----	----

* Dequeue from Queue

```
int Dequeue () {
    if (isEmpty) {
        S.O.P("underflow");
    }
    int removed = Queue[front++];
    return removed;
}
front = -1, rear = 3
```

① removed = Queue[0]
10 is delete

② removed = Queue[1]
20 is delete

③ removed = Queue[2]
30 is delete

④ removed = Queue[3]
40 is delete.

* Display elements of Queue.

↓ front
↓ rear

10	20	30	40
----	----	----	----

```
void display () {
    for (int i = front; i <= rear; i++) {
        S.O.P(Queue[i]);
    }
}
```

front = -1, rear = 3

0 <= 3 ✓ ~~✗~~ O/P ⇒ 10

1 <= 3 ✓ ~~✗~~ O/P ⇒ 20

2 <= 3 ✓ ~~✗~~ O/P ⇒ 30

3 <= 3 ✓ ~~✗~~ O/P ⇒ 40